Bazy Danych – projekt (prowadzący: mgr inż. Michał Jaroszczuk)

Gdziewypije – raport z etapu 2

Franciszek Pora, Jakub Konieczko | grupa 1, WT TP 15:15 | Wrocław, 27.10.2026

1 Technologie

Typ aplikacji	webowa (strona w przeglądarce)
Typ bazy danych	relacyjna, PostgreSQL (z PostGIS do zapytań po odległości)
Frontend	React, Leaflet (mapa)
Backend	Python, FastAPI
Dane z zewnątrz	Overpass API (OpenStreetMap)
Narzędzia	Git i GitHub

2 Model danych

2.1 Opis tabel

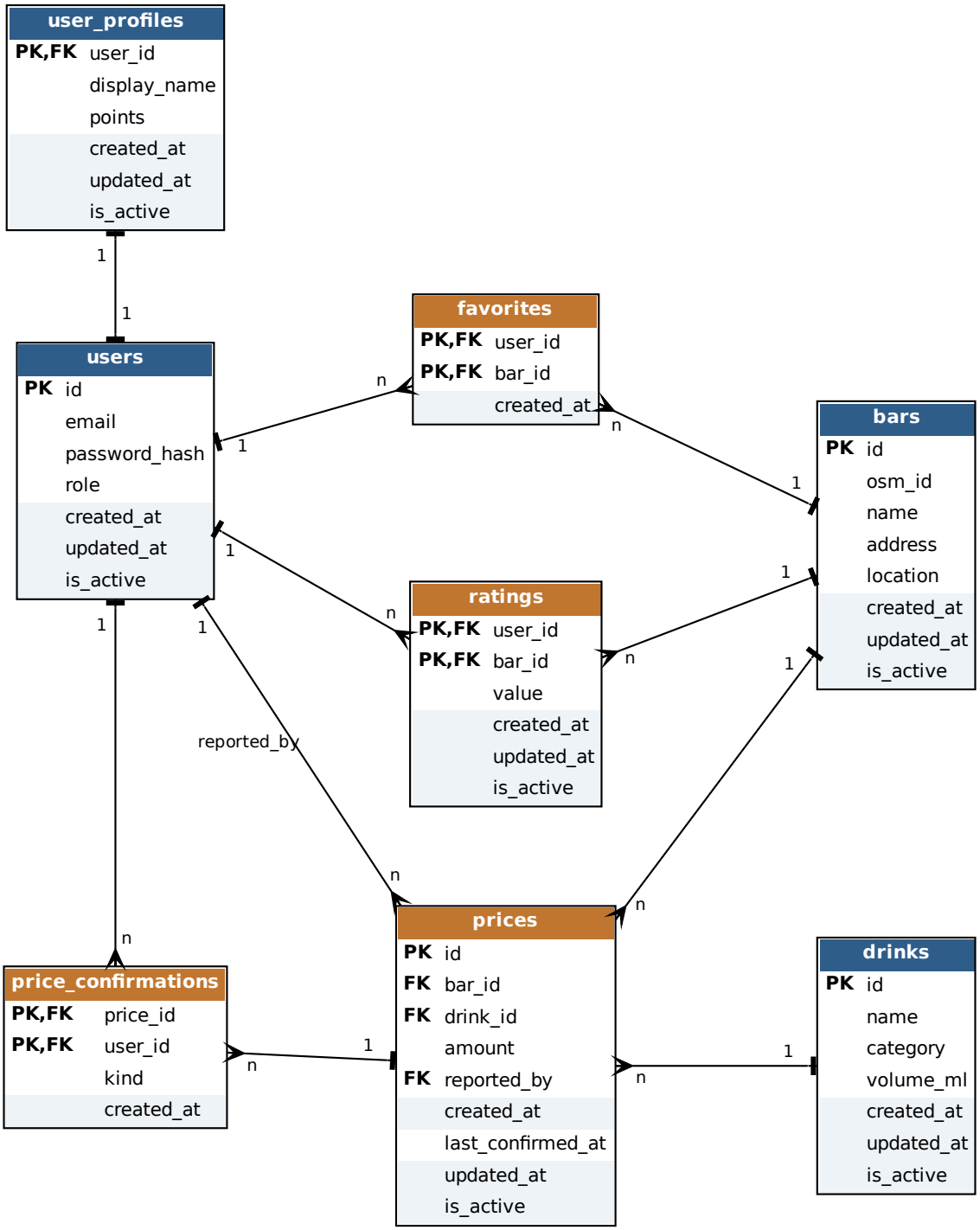
Tabela	Opis
users	konta: e-mail (unikalny), hash hasła, rola (user lub admin)
user_profiles	nazwa wyświetlana i punkty użytkownika (relacja 1:1 z users)
bars	bary z OpenStreetMap: nazwa, adres, godziny otwarcia, lokalizacja
drinks	katalog napojów (piwo, drinki, shoty, wino) z pojemnością
prices	ceny napojów w barach. Łączy bary z napojami: bar ma napój w ofercie, gdy ma dla niego cenę. Trzymamy historię, aktualna cena to najnowszy wpis
price_confirmations	potwierdzenia cen przez użytkowników (jedno na osobę), z typem: potwierdza albo zmiana
ratings	oceny barów 1–5 (jedna ocena użytkownika na bar)
favorites	ulubione bary użytkownika

2.2 Relacje i normalizacja

- **1:1** – users i user_profiles.
- **1:n** – users → prices (kto zgłosił cenę), prices → price_confirmations.
- **n:n** – bars i drinks przez prices; users i bars przez ratings oraz favorites; users i prices przez price_confirmations.
- **3NF:** każda informacja jest tylko w jednym miejscu, np. nazwa napoju jest w drinks, a w prices tylko jego numer.
- **Odstępstwa:** data potwierdzenia ceny i punkty dałoby się policzyć, ale dla wygody trzymamy je gotowe.
- **Pola audytowe** (jasne tło na diagramie): created_at, updated_at i is_active. Dane wyłączamy (is_active = false) zamiast usuwać. Wyjątek to ulubione, gdzie usuwamy wiersz. W ulubionych i potwierdzeniach jest tylko created_at.

2.3 Diagram ERD

Baza będzie miała 8 tabel. Na diagramie są najważniejsze pola (PK – klucz główny, FK – klucz obcy). Jasne tło oznacza pola audytowe (created_at, updated_at, is_active), a pomarańczowy nagłówek tabelę łączącą (relacja n:n). Kilka kolumn z „PK,FK” w jednej tabeli tworzy razem jeden klucz złożony.



Rysunek 1: ERD bazy Gdziewypije.

3 Widoki i funkcje

Zaplanowaliśmy 3 widoki, czyli zapisane zapytania, z których korzysta mapa i ranking.

Widok	Do czego
v_bar_stats	średnia ocena i ocena ważona liczbą głosów (bar z jedną piątką nie wyprzedzi baru z wieloma dobrymi)
v_current_prices	aktualna cena napoju w barze i oznaczenie „do weryfikacji” po 90 dniach bez potwierdzenia
v_top_month	top miesiąca: oceny dodane lub zmienione w tym miesiącu plus dodania do ulubionych (minimum 3 zdarzenia), przy remisie decyduje średnia ocena

Do zgłaszania i potwierdzania zrobimy też 3 funkcje w bazie. Robią to samo co transakcje T1–T3, ale jednym wywołaniem.

Funkcja	Do czego	Transakcja
register_user	tworzy konto i profil	T1
report_price	zapisuje cenę i daje 5 punktów	T2
confirm_price	zapisuje potwierdzenie, odświeża datę ceny i daje 1 punkt	T3

4 Transakcje

Transakcja to kilka zapytań, które wykonują się razem albo wcale (BEGIN – start, COMMIT – zatwierdzenie).

T1 – rejestracja (WB.04)

Konto i profil zapisują się razem.

```
BEGIN;
  INSERT INTO users (email, password_hash) VALUES ('nowy@test.pl', 'hash');
  INSERT INTO user_profiles (user_id, display_name) VALUES (7, 'Nowy');
COMMIT;
```

T2 – zgłoszenie ceny (WB.06)

Cena i 5 punktów dla autora zapiszą się razem.

```
BEGIN;
  INSERT INTO prices (bar_id, drink_id, amount, reported_by) VALUES (2, 1, 11.50, 3);
  UPDATE user_profiles SET points = points + 5 WHERE user_id = 3;
COMMIT;
```

T3 – potwierdzenie ceny (WB.07)

Potwierdzenie, nowa data ceny i 1 punkt. Ta sama osoba nie potwierdzi dwa razy (klucz główny).

```
BEGIN;
  INSERT INTO price_confirmations (price_id, user_id) VALUES (9, 4);
  UPDATE prices SET last_confirmed_at = now() WHERE id = 9;
  UPDATE user_profiles SET points = points + 1 WHERE user_id = 4;
COMMIT;
```

T4 – ocena baru (WB.05)

Dodaje ocenę albo zmienia poprzednią.

```
BEGIN;
  INSERT INTO ratings (user_id, bar_id, value) VALUES (6, 2, 5)
  ON CONFLICT (user_id, bar_id) DO UPDATE SET value = 5;
COMMIT;
```

T5 – ulubione (WB.08)

Dodanie do ulubionych i usunięcie.

```
BEGIN;
  INSERT INTO favorites (user_id, bar_id) VALUES (6, 3);
COMMIT;
BEGIN;
  DELETE FROM favorites WHERE user_id = 6 AND bar_id = 3;
COMMIT;
```

5 Zapytania zaawansowane

Zaplanowaliśmy 3 zapytania trochę bardziej rozbudowane niż zwykły SELECT.

Q1 – bary w promieniu 2 km (WB.10)

Bary w promieniu 2 km od przykładowego punktu (współrzędne w okolicy Rynku) z oceną i odległością. 2000 to promień w metrach (PostGIS i JOIN).

```
SELECT b.name, s.avg_rating,
       ST_Distance(b.location, ST_MakePoint(17.032, 51.110)::geography) AS odleglosc_m
FROM bars b
JOIN v_bar_stats s ON s.bar_id = b.id
WHERE ST_DWithin(b.location, ST_MakePoint(17.032, 51.110)::geography, 2000)
ORDER BY odleglosc_m;
```

Q2 – ranking barów (WB.03)

Ranking według oceny ważonej (funkcja okna RANK).

```
SELECT RANK() OVER (ORDER BY s.weighted_score DESC) AS miejsce,
       b.name, s.avg_rating, s.ratings_count, s.weighted_score
FROM bars b
JOIN v_bar_stats s ON s.bar_id = b.id
WHERE s.ratings_count > 0
ORDER BY miejsce;
```

Q3 – top 3 miesiąca (WB.09)

Korzysta z widoku v_top_month, w którym jest GROUP BY i HAVING.

```
SELECT name, activity, avg_rating FROM v_top_month LIMIT 3;
```

6 Prognoza użycia tabel

Liczby są szacunkowe. Barów jest ok. 160 (tyle pokazał Overpass).

Tabela	Odczyt	Zapis	Wierszy
users, user_profiles	rzadko	rzadko	100–1000
bars	bardzo często	rzadko (import)	150–300
drinks	często	prawie wcale	10–20
prices	bardzo często	średnio	1000–3000
price_confirmations	rzadko	średnio	1000–5000
ratings	często	średnio	1000–5000
favorites	średnio	średnio	500–2000

Baza będzie głównie do odczytu. Historii cen nie usuwamy, więc prices urośnie najbardziej.

7 Wymagania funkcjonalne i niefunkcjonalne

WF to to, co system robi, a WNF to jaki ma być. Numery WB są z etapu 1.

WB	Wymagania funkcjonalne (WF)	Niefunkcjonalne (WNF)
WB.01	WF.01 System musi pokazywać bary na mapie. WF.02 System musi pokazywać po kliknięciu ocenę i ceny baru.	WNF.01 Mapa musi załadować się w mniej niż 3 sekundy.
WB.02	WF.03 System musi pozwalać przełączać mapę między ceną a oceną. WF.04 System musi pokazywać „brak danych” przy barze bez ceny.	–
WB.03	WF.05 System musi pokazywać listę barów z rankingiem. WF.06 System musi pozwalać szukać barów po nazwie.	–
WB.04	WF.07 System musi pozwalać założyć konto. WF.08 System musi pozwalać się zalogować.	WNF.02 Hasła muszą być zapisane jako hash. WNF.03 Założenie konta musi zająć mniej niż minutę.
WB.05	WF.09 System musi pozwalać ocenić bar (1–5), jedna ocena na bar.	–
WB.06	WF.10 System musi pozwalać zgłosić cenę napoju. WF.11 System musi odrzucać ceny nie większe od 0.	–
WB.07	WF.12 System musi pozwalać potwierdzić cenę. WF.13 System musi oznaczać cenę „do weryfikacji” po 90 dniach bez potwierdzenia.	–
WB.08	WF.14 System musi pozwalać dodawać bary do ulubionych i je usuwać. WF.15 System musi pozwalać pokazać tylko ulubione.	–
WB.09	WF.16 System musi pokazywać 3 najpopularniejsze bary miesiąca.	WNF.04 Ranking musi liczyć się na bieżąco.
WB.10	WF.17 System musi pokazywać bary w pobliżu użytkownika.	WNF.05 Aplikacja musi działać też bez zgody na lokalizację.
WB.11	WF.18 Administrator musi móc dodawać, zmieniać i wyłączać bary i napoje.	WNF.06 Dostęp tylko dla administratora.
WB.12	WF.19 System musi dawać punkty za zgłoszenie i potwierdzenie ceny.	–
WB.13	nie robimy (Won’t)	–

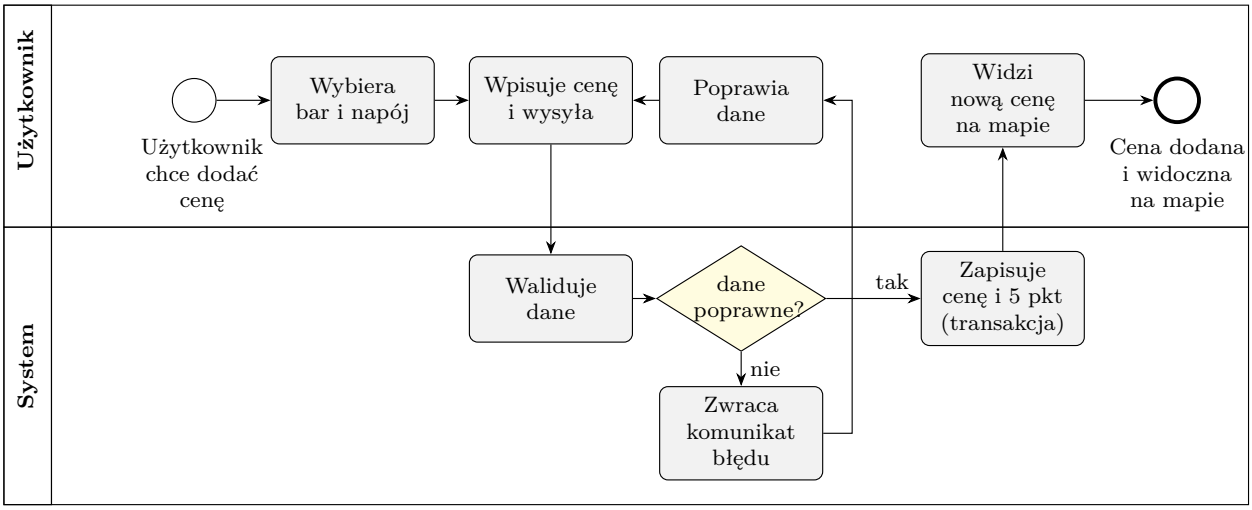
Wymagania niefunkcjonalne dla całego systemu (nie dotyczą jednego WB):

- WNF.07 Strona musi działać na telefonie.
- WNF.08 Dane muszą być spójne (transakcje).
- WNF.09 Strona musi podawać źródło danych (OpenStreetMap).
- WNF.10 Import barów nie może robić duplikatów.
- WNF.11 Tabele muszą mieć pola audytowe.
- WNF.12 System musi obsłużyć co najmniej 1000 kont i 300 barów.

8 Diagramy

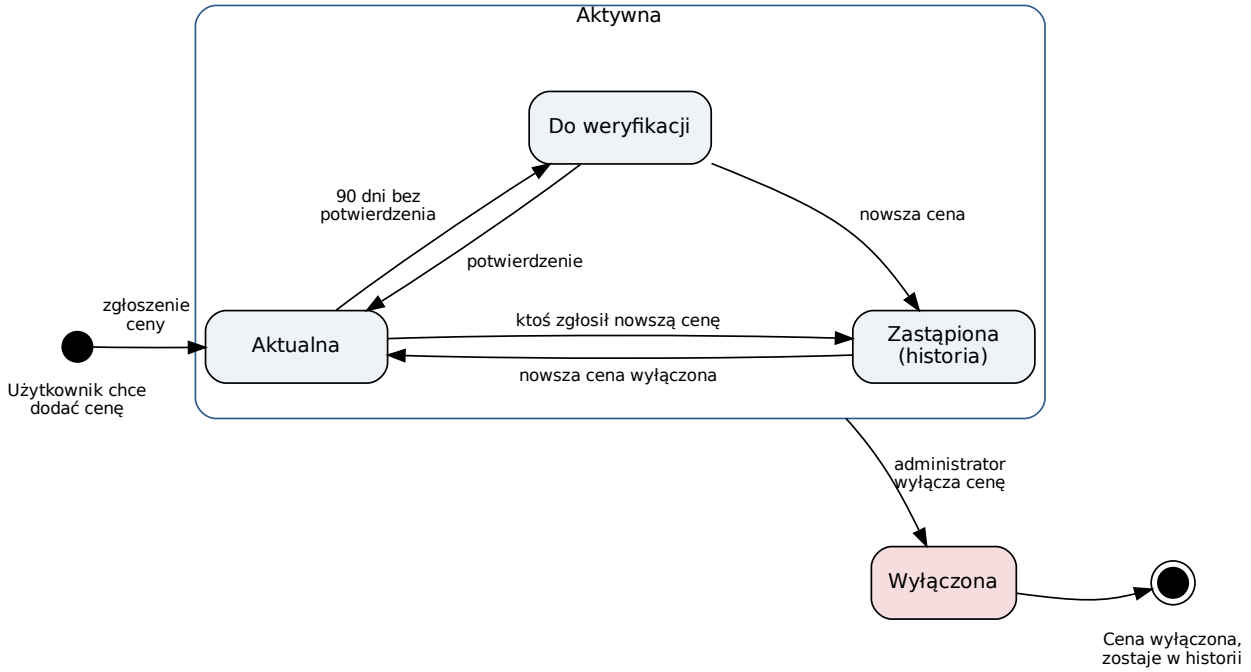
Wybraliśmy diagram BPMN (proces dodania ceny) i diagram stanów UML (cykl życia ceny).

8.1 Diagram BPMN: dodanie ceny



Rysunek 2: BPMN: dodanie ceny napoju (dwa tory: użytkownik i system).

8.2 Diagram stanów (UML): cena



Rysunek 3: Cykl życia ceny napoju. To nie jest osobna kolumna: stan wynika z is_active, last_confirmed_at i tego, czy jest nowsza cena.

9 Opisy przypadków użycia

To cztery najważniejsze przypadki z diagramu przypadków użycia z etapu 1.

PU.01 Rejestracja (WB.04, HU.04)	
Aktor	gość
Ścieżka główna	1. Gość wybiera „Załóż konto”. 2. Wpisuje e-mail, hasło i nazwę. 3. System sprawdza dane. 4. System zapisuje konto i profil (T1). 5. Gość jest zalogowany.
Ścieżki alternatywne	3a. E-mail jest już w bazie: komunikat „konto istnieje”. 3b. Nazwa krótsza niż 2 znaki: komunikat przy polu.

PU.02 Zgłoszenie ceny (WB.06, HU.06)	
Aktor	użytkownik zalogowany
Ścieżka główna	1. Użytkownik otwiera bar na mapie. 2. Wybiera napój i wpisuje cenę. 3. System sprawdza dane. 4. System zapisuje cenę i dodaje 5 punktów (T2). 5. Nowa cena pojawia się na mapie.
Ścieżki alternatywne	1a. Użytkownik nie jest zalogowany: przejście do logowania. 3a. Cena nie jest większa od 0: komunikat, nic się nie zapisuje. 3b. Bar został wyłączony: błąd.

PU.03 Potwierdzenie ceny (WB.07, HU.07)	
Aktor	użytkownik zalogowany
Ścieżka główna	1. Użytkownik widzi cenę (może mieć etykietę „do weryfikacji”). 2. Wybiera „Zgadza się”. 3. System zapisuje potwierdzenie i dodaje 1 punkt (T3). 4. Etykieta znika.
Ścieżki alternatywne	2a. Użytkownik już potwierdzał tę cenę: komunikat „już potwierdzone”. 2b. Cena jest inna: użytkownik wybiera „zmiana” i zgłasza nową cenę (PU.02).

PU.04 Ocena baru (WB.05, HU.05)	
Aktor	użytkownik zalogowany
Ścieżka główna	1. Użytkownik wybiera od 1 do 5 gwiazdek. 2. System zapisuje ocenę (T4). 3. Średnia i ranking się aktualizują.
Ścieżki alternatywne	2a. Użytkownik ocenił już ten bar: nowa ocena zastępuje starą. 2b. Wartość spoza 1–5: błąd.